

Unit III – Software Project Management

1. Introduction to Software Project Management

1.1 What is Software Project Management?

Software Project Management (SPM) is the process of planning, organizing, monitoring, and controlling a software project so that it is completed:

- within the planned time
- within the planned budget
- with the required quality
- according to the customer's requirements

software project management means managing all the activities and resources required to develop software successfully.

1.2 Why is Software Project Management Needed?

Software projects involve many activities such as requirement analysis, design, coding, testing, and maintenance. These activities need proper coordination.

Software project management helps to:

1. Complete the project on time
2. Control project cost
3. Manage project resources
4. Maintain software quality
5. Identify and manage risks
6. Coordinate team members
7. Track project progress
8. Satisfy customer requirements

1.3 Objectives of Software Project Management

The main objectives are:

1. Time Management

To complete the project within the scheduled time.

2. Cost Management

To complete the project within the approved budget.

3. Quality Management

To ensure that the software meets the required quality standards.

4. Resource Management

To use people, hardware, software, tools, and other resources effectively.

5. Risk Management

To identify possible problems and take suitable actions to reduce their impact.

6. Scope Management

To clearly define what is included and what is not included in the project.

1.4 Major Activities of Software Project Management

Software project management mainly includes:

1. Project Planning
 - Define project objectives.
 - Identify required activities.
 - Prepare a project plan.
2. Project Estimation
 - Estimate the required effort, time, cost, and resources.
3. Project Scheduling
 - Decide when each activity will start and finish.
 - Set deadlines for different tasks.
4. Project Organization
 - Define roles and responsibilities.
 - Form the project team.
5. Risk Management
 - Identify possible risks.
 - Analyze their effect.
 - Prepare solutions.
6. Project Monitoring and Control
 - Check the actual progress of the project.
 - Compare it with the planned progress.
 - Take corrective actions when required.
7. Quality Management
 - Ensure that the software follows required quality standards.
8. Configuration Management
 - Manage changes made to software, documents, and other project items.

1.5 Software Project Manager

A Software Project Manager is responsible for managing the software project and ensuring its successful completion.

Main responsibilities include:

- Planning the project
- Estimating time and cost
- Allocating resources
- Managing the project team
- Managing risks
- Monitoring project progress
- Communicating with customers and stakeholders
- Ensuring software quality
- Handling changes in the project

Example

Suppose a college wants to develop a Student Attendance Management System.

The project manager needs to:

- decide what features are required
- estimate development time and cost
- assign developers and testers
- prepare a project schedule
- identify risks
- monitor development progress
- ensure that the system is delivered on time and works correctly

Thus, managing all these activities is called Software Project Management.

Software Project Management is the systematic planning, organizing, monitoring, and controlling of software development activities to complete a project within its planned time, cost, scope, and quality.

Remember the four important aspects: Time + Cost + Scope + Quality.

2. Selection of a Project Approach

2.1 What is a Project Approach?

A project approach is the method or process model selected to develop a software system.

It defines how the software development activities will be planned, performed, and managed from the beginning to the completion of the project.

Project approach means choosing the most suitable software development model for a particular project.

Examples include:

- Waterfall Model
- Incremental Model
- Prototyping Model
- Spiral Model
- Agile Model

2.2 Need for Selecting a Project Approach

Different software projects have different requirements. Therefore, the same development approach may not be suitable for every project.

The project approach is selected to:

1. Understand project requirements
2. Manage project risks
3. Control development cost
4. Complete the project within the required time
5. Handle changes in requirements
6. Maintain software quality
7. Improve communication with customers
8. Use available resources effectively

2.3 Factors Affecting Selection of Project Approach

The following factors should be considered:

1. Requirements

- If requirements are clear and stable, Waterfall may be suitable.
- If requirements are likely to change frequently, Agile may be better.

2. Project Size

- Small projects may use simple and flexible approaches.
- Large and complex projects may require more structured approaches.

3. Project Complexity

Highly complex projects may require an approach that supports continuous risk analysis and evaluation.

4. Risk

If the project has high technical or business risks, a **risk-driven approach such as Spiral** may be considered.

5. Customer Involvement

If regular customer feedback is required, **Agile or Prototyping** can be useful.

6. Time Available

If software needs to be delivered in small releases, an **Incremental or Agile approach** may be suitable.

7. Team Experience

The skills and experience of the development team should also be considered.

8. Technology

If new or unfamiliar technology is being used, an approach that allows experimentation and prototyping may be preferred.

2.4 Common Project Approaches

Approach	Suitable When
Waterfall	Requirements are clear and stable
Incremental	Software can be delivered in parts
Prototyping	Requirements are unclear
Spiral	Project has significant risks
Agile	Requirements change frequently and customer feedback is important

Example

Suppose a college wants to develop a **Student Management System**.

If all requirements are already fixed and unlikely to change, the **Waterfall Model** may be selected.

However, if the college wants to regularly add or modify features based on feedback, an **Agile approach** may be more suitable.

Selection of a project approach is the process of choosing the most suitable software development model based on requirements, project size, complexity, risk, time, resources, and customer involvement.

3. Project Estimation Techniques

3.1 What is Project Estimation?

Software project estimation is the process of predicting the **effort, time, cost, and resources** required to complete a software project.

Project estimation means finding out how much time, money, effort, and resources are needed to develop the software.

3.2 What is Estimated?

A software project manager generally estimates:

1. **Effort** – How much work is required?
2. **Time** – How long will the project take?
3. **Cost** – How much will the project cost?
4. **Resources** – How many people, tools, and hardware resources are required?

3.3 Why is Project Estimation Important?

Project estimation helps to:

- Prepare a realistic **project schedule**
- Prepare the **project budget**
- Decide the required **team size**
- Allocate resources properly
- Set realistic deadlines
- Identify possible problems early
- Monitor project progress
- Support project planning and decision-making

3.4 Common Project Estimation Techniques

1. Expert Judgment

Estimation is made using the **knowledge and experience of experts**.

Example:

An experienced project manager estimates that a student management system will require approximately 4 months.

Advantage: Simple and quick.

Limitation: Depends heavily on the expert's experience.

2. Analogy-Based Estimation

The current project is compared with a **similar completed project**.

Example:

If a previous attendance system took 3 months, a similar new system may also require approximately 3–4 months.

Advantage: Easy to understand.

Limitation: A suitable similar project may not always be available.

3. Algorithmic / Parametric Estimation

A mathematical formula or model is used to calculate the estimated effort or cost.

A well-known example is the **COCOMO (Constructive Cost Model)** developed by **Barry Boehm**.

A basic COCOMO relationship is:

$$\text{Effort} = a \times (\text{KLOC})^b$$

Where:

- **Effort** = estimated effort in person-months
- **KLOC** = Thousand Lines of Code
- **a, b** = constants depending on the project type

4. Top-Down Estimation

The **total project effort or cost is estimated first** and then divided among different activities.

Example:

Total estimated effort = 100 person-months

This effort is then distributed among:

- Requirements
- Design
- Coding
- Testing
- Maintenance

Advantage: Quick and useful during early planning.

5. Bottom-Up Estimation

Each individual task is estimated separately and then all estimates are added to obtain the total project estimate.

Example

Suppose a college wants to develop an **Online Examination System**.

The project manager estimates:

- Requirements → 2 weeks
- Design → 3 weeks
- Coding → 6 weeks
- Testing → 3 weeks
- Deployment → 1 week

Based on these estimates, the project manager can prepare the **schedule, budget, and resource plan**.

Project estimation is the process of predicting the effort, time, cost, and resources required to complete a software project.

4. Project Planning and Project Scheduling

4.1 Project Planning

Project planning is the process of deciding **what work has to be done, how it will be done, who will do it, and what resources are required** to complete a software project successfully.

Project planning is preparing a roadmap for completing the software project.

4.2 Objectives of Project Planning

Project planning helps to:

1. Define the **project scope and objectives**.
2. Identify the **activities and tasks**.
3. Estimate **time, cost, and effort**.
4. Identify required **resources**.
5. Assign responsibilities to team members.
6. Identify possible **risks**.
7. Establish a proper **schedule**.
8. Provide a basis for **monitoring and controlling** the project.

4.3 Major Activities in Project Planning

1. Define Project Scope

Identify what the project will and will not include.

2. Identify Tasks

Break the project into smaller activities such as:

- Requirement analysis
- Design
- Coding
- Testing
- Deployment

3. Estimate Resources

Determine the required:

- People
- Hardware
- Software
- Budget

4. Estimate Time and Cost

Estimate how much time and money are required for each activity.

5. Assign Responsibilities

Assign specific tasks to appropriate team members.

6. Identify Risks

Identify possible problems that may affect the project.

4.4 Project Scheduling

Project scheduling is the process of deciding **when each project activity will start and finish**.

In simple words:

Scheduling tells us when each task should be performed and how long it should take.

4.5 Objectives of Project Scheduling

- Complete the project on time.
- Define start and finish dates.
- Allocate resources effectively.
- Identify dependencies between tasks.
- Monitor project progress.
- Avoid unnecessary delays.

4.6 Steps in Project Scheduling

Step 1: Identify Activities

List all activities required to complete the project.

Step 2: Determine Activity Sequence

Decide which activities must be completed before others.

Step 3: Estimate Duration

Determine the time required for each activity.

Step 4: Assign Resources

Assign team members and other resources to activities.

Step 5: Prepare the Schedule

Set the start and completion dates.

Step 6: Monitor the Schedule

Compare actual progress with the planned schedule and take corrective action when required.

Example

Suppose a **Library Management System** is being developed:

Activity	Duration
Requirement Analysis	1 week
System Design	2 weeks
Coding	4 weeks
Testing	2 weeks
Deployment	1 week

The project manager uses this information to prepare the project schedule and determine the expected completion date.

4.8 Project Planning vs Project Scheduling

Project Planning	Project Scheduling
Decides what needs to be done	Decides when it will be done
Defines activities, resources, cost, etc.	Defines start and finish dates
Broader activity	Part of project planning
Creates the project roadmap	Creates the project timetable

Project Planning defines the work, resources, cost, and approach required for a project, while **Project Scheduling** determines the sequence and timing of project activities.

5. Project Organization and Team Structures

5.1 Project Organization

Project organization is the process of arranging the **people, roles, responsibilities, and communication** needed to complete a software project.

Project organization defines who will do what work and who will be responsible for the project.

A good project organization helps the team work together effectively and complete the project successfully.

5.2 Objectives of Project Organization

The main objectives are:

1. Define **roles and responsibilities**.
2. Establish proper **communication** among team members.
3. Assign tasks according to **skills and experience**.
4. Coordinate different project activities.
5. Improve **team productivity**.
6. Ensure proper **decision-making**.
7. Complete the project within the planned **time and cost**.

5.3 Common Roles in a Software Project

1. Project Manager

Responsible for overall project planning, scheduling, monitoring, and control.

2. Business/System Analyst

Studies customer requirements and prepares the software requirements.

3. Software Designer

Designs the architecture and structure of the software.

4. Developer/Programmer

Writes and maintains the program code.

5. Tester

Tests the software and identifies defects.

6. Database Administrator

Manages the database and ensures proper data storage and access.

7. Customer/Client

Provides requirements and gives feedback on the software.

5.4 Software Team Structures

A **team structure** defines how team members are organized and how they communicate with each other.

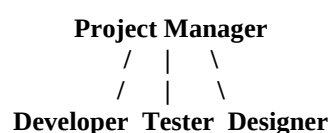
The commonly used structures are:

1. Controlled Centralized Structure

In this structure, **one person or a small group controls most decisions.**

- Project manager has major responsibility.
- Communication mainly follows a central authority.
- Suitable when quick decisions are required.

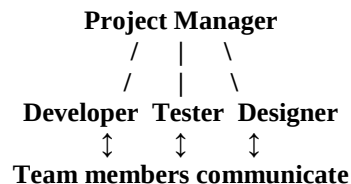
Example:



2. Controlled Decentralized Structure

In this structure, the project manager provides overall direction, but **team members also participate in decision-making**.

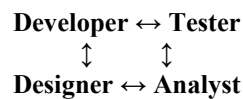
- Some decisions are centralized.
- Team members have more responsibility.
- Communication is more flexible.



3. Democratic Decentralized Structure

In this structure, **decisions are made collectively by team members**.

- There is no strong central authority.
- Team members communicate directly with each other.
- Suitable for experienced teams and complex problem-solving.



5.5 Factors Affecting Team Structure

The choice of team structure depends on:

- **Project size**
- **Project complexity**
- **Team size**
- **Team members' experience**
- **Type of project**
- **Communication requirements**
- **Project deadlines**

5.6 Importance of Proper Team Structure

A suitable team structure helps to:

- Improve communication
- Reduce conflicts
- Clearly define responsibilities
- Improve productivity
- Make decision-making easier
- Complete the project efficiently

Project organization defines the roles, responsibilities, and relationships among project members, while team structure defines how these members communicate and make decisions.

6. Risk Management

6.1 What is Risk?

A **risk** is a possible problem or uncertain event that may negatively affect a software project in the future.

Risk is something that may happen and can cause loss, delay, extra cost, or failure of the project.

Examples:

- A key developer may leave the project.
- Software requirements may change.
- The project may exceed its budget.
- A technology may not work as expected.

6.2 What is Risk Management?

Risk management is the process of **identifying, analyzing, planning, and controlling risks** throughout the software project.

The main goal is to **reduce the possibility and impact of risks.**

6.3 Objectives of Risk Management

1. Identify possible risks early.
2. Analyze the probability and impact of risks.
3. Prepare plans to handle risks.
4. Reduce project losses.
5. Minimize project delays and cost overruns.
6. Improve the chances of project success.

6.4 Types of Software Risks

1. Project Risks

These risks affect the **project schedule, resources, or cost**.

Examples:

- Lack of skilled developers
- Incorrect cost estimation
- Delay in project activities
- Team member leaving the project

2. Technical Risks

These risks are related to **technology, design, or implementation**.

Examples:

- New technology does not work as expected.
- Performance problems
- Integration problems
- Difficulties in implementing a feature

3. Business Risks

These risks affect the **business or organization**.

Examples:

- Change in business requirements
- Loss of funding
- Product may not meet market needs
- Customer may cancel the project

6.5 Risk Management Process

Risk management generally involves the following steps:



1. Risk Identification

Identify all possible risks that may affect the project.

Example:

A project depends on a particular third-party software service.

2. Risk Analysis

Analyze each risk based on:

- **Probability** – How likely is the risk to occur?
- **Impact** – How seriously will it affect the project?

3. Risk Planning

Prepare suitable actions to handle the identified risks.

For example, if a key developer may leave, another developer can be trained as a backup.

4. Risk Monitoring

Regularly observe identified risks and check whether their probability or impact has changed.

5. Risk Control

Take corrective action when a risk occurs or becomes more serious.

6.6 Risk Response Strategies

Common strategies are:

1. Avoidance

Change the project plan to completely avoid the risk.

2. Reduction/Mitigation

Take actions to reduce the probability or impact of the risk.

3. Transfer

Transfer the responsibility or impact of the risk to another party.

4. Acceptance

Accept the risk when its impact is small or cannot be avoided.

Example

Suppose a project depends on a **single developer** who has specialized technical knowledge.

Risk: The developer may leave the project.

Impact: Project may be delayed.

Risk response: Train another team member and maintain proper documentation.

Thus, the project can continue even if the original developer leaves.

Risk management is the systematic process of identifying, analyzing, planning, monitoring, and controlling risks to reduce their effect on a software project.

7. Resource Allocation

7.1 What is Resource Allocation?

Resource allocation is the process of **assigning the required resources to different activities of a software project.**

Resource allocation means deciding what resources are needed, how much is needed, and which task will use them.

7.2 Types of Project Resources

Software projects mainly require the following resources:

1. Human Resources

People involved in the project.

Examples:

- Project Manager
- Software Developer
- Software Tester
- System Analyst
- Designer
- Database Administrator

2. Hardware Resources

Physical equipment required for development.

Examples:

- Computers

- Servers
- Network devices
- Testing devices

3. Software Resources

Software tools required for the project.

Examples:

- Programming languages
- IDEs
- Database software
- Testing tools
- Version control tools

4. Financial Resources

Money required to complete the project.

Examples:

- Employee salaries
- Software licenses
- Hardware purchase
- Cloud services
- Training costs

5. Time

Time is also an important project resource because every activity has a specific duration and deadline.

7.3 Objectives of Resource Allocation

The main objectives are:

1. **Use resources efficiently.**
2. Assign the **right person to the right task.**
3. Avoid unnecessary resource usage.
4. Prevent overloading of team members.
5. Complete tasks within the planned schedule.
6. Control project cost.
7. Ensure that required resources are available when needed.

7.4 Steps in Resource Allocation

Step 1: Identify Required Resources

Determine the resources needed for each project activity.

Step 2: Determine Resource Availability

Check which resources are currently available.

Step 3: Assign Resources

Assign suitable people, tools, hardware, and budget to each activity.

Step 4: Prepare Resource Schedule

Decide **when and for how long** each resource will be required.

Step 5: Monitor Resource Usage

Regularly check whether resources are being used according to the plan.

Step 6: Reallocate Resources

If a resource is overloaded or unavailable, assign another suitable resource.

Example

Consider an **Online Examination System** project:

Activity	Resources
Requirement Analysis	System Analyst
UI Design	UI Designer
Coding	2 Developers
Database Development	Database Administrator
Testing	2 Testers
Deployment	Developer + System Administrator

Here, the project manager allocates the required people and resources to each activity.

7.6 Problems in Resource Allocation

Poor resource allocation can result in:

- Project delays
- Increased cost
- Employee overload
- Underutilization of resources
- Poor software quality
- Failure to meet deadlines

Resource allocation is the process of identifying, assigning, scheduling, and managing human, hardware, software, financial, and time resources to complete a software project efficiently.

8. Project Monitoring and Control

8.1 What is Project Monitoring and Control?

Project monitoring and control is the process of **regularly checking the progress of a software project and taking corrective actions when the actual progress differs from the planned progress.**

Monitoring tells us what is happening in the project, while control helps us take action when something goes wrong.

8.2 Need for Project Monitoring and Control

It helps to:

1. Check whether the project is progressing **according to the plan.**
2. Identify **delays and problems** early.
3. Control project **cost and schedule.**
4. Monitor the use of **resources.**
5. Ensure the required **quality.**
6. Identify and manage **risks.**
7. Take corrective actions when necessary.
8. Ensure that project objectives are achieved.

8.3 Project Monitoring

Project monitoring means continuously collecting and reviewing information about the project.

The project manager monitors:

- Task completion
- Project schedule
- Project cost
- Resource usage
- Software quality
- Risks
- Team performance

Example:

If coding was planned to be completed in 4 weeks but only 60% is completed after 4 weeks, the project manager identifies the delay through monitoring.

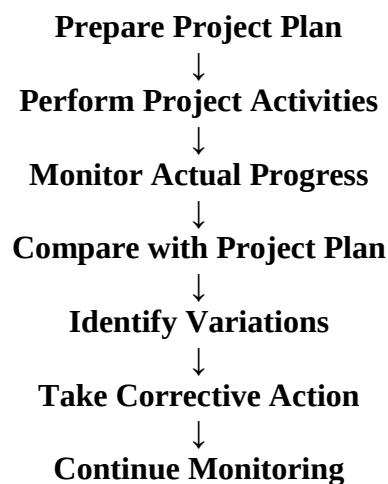
8.4 Project Control

Project control means taking appropriate action when there is a difference between **planned** and **actual performance**.

Examples of corrective actions:

- Assign additional developers.
- Change task priorities.
- Modify the project schedule.
- Allocate additional resources.
- Reduce unnecessary activities.
- Take action to control additional costs.

8.5 Monitoring and Control Process



8.6 Important Areas to Monitor

1. Schedule Monitoring

Check whether activities are completed according to the planned schedule.

2. Cost Monitoring

Compare actual project expenses with the planned budget.

3. Resource Monitoring

Check whether human, hardware, and software resources are being used effectively.

4. Quality Monitoring

Check whether the software meets the required quality standards.

5. Risk Monitoring

Track identified risks and check for new risks.

6. Scope Monitoring

Ensure that unnecessary features or requirements are not added without proper approval.

8.7 Example

Suppose a project is planned to cost **₹5 lakh** and take **6 months**.

After 4 months:

- Planned progress = 70%
- Actual progress = 50%

The project manager identifies a **20% delay** and investigates the reason.

If the delay is due to insufficient developers, additional developers may be assigned.

This is an example of **project monitoring and control**.

Project monitoring and control is the continuous process of measuring project progress, comparing actual performance with the plan, identifying deviations, and taking corrective actions to keep the project on track.

9. Software Configuration Management

9.1 What is Software Configuration Management?

Software Configuration Management (SCM) is the process of **identifying, organizing, controlling, and tracking changes made to software and related documents throughout the software development life cycle.**

SCM helps to manage different versions of software and control changes made during development.

For example, when developers modify source code several times, SCM helps maintain and identify the different versions of that code.

9.2 Need for Software Configuration Management

SCM is needed because software undergoes many changes during development.

It helps to:

1. **Control changes** made to software.
2. Maintain different **versions** of software.
3. Prevent unauthorized changes.
4. Track **who made a change and when**.
5. Avoid conflicts when multiple developers work together.
6. Maintain the **integrity and consistency** of software.
7. Easily restore an earlier version when required.

9.3 Software Configuration Item (SCI)

A **Software Configuration Item (SCI)** is any software-related item that needs to be controlled and managed.

Examples:

- Source code
- Requirement documents
- Design documents
- Test cases
- User manuals
- Configuration files
- Database scripts

9.4 Main Activities of Software Configuration Management

1. Configuration Identification

Identify and name the items that need to be controlled.

Example: Source code, SRS document, test cases, etc.

2. Version Control

Maintain different versions of software and documents.

Example:

Version 1.0 → Initial Release

Version 1.1 → Bug Fixes

Version 2.0 → New Features

3. Change Control

Changes to the software are reviewed and approved before they are implemented.

Example:

A customer requests a new login feature. The change is evaluated before developers modify the system.

4. Configuration Status Accounting

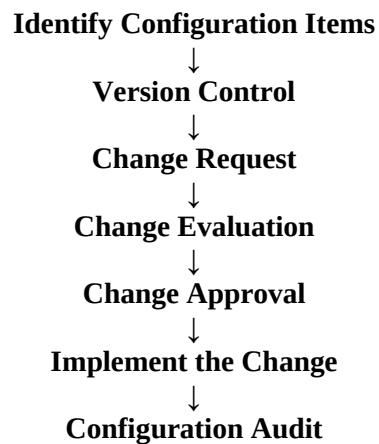
Maintain records of:

- Current version
- Changes made
- Change requests
- Approval status

5. Configuration Auditing

Check whether the software and its configuration items are consistent with the approved requirements and standards.

9.5 Simple SCM Process



9.6 Example

Suppose three developers are working on a **Student Management System**.

Developer 1 modifies the login module, Developer 2 modifies the attendance module, and Developer 3 modifies the database.

Without SCM, their changes may conflict with each other.

Using a **version control system**, each change can be tracked and managed properly.

9.7 Benefits of SCM

- Better control over software changes
- Easy version management
- Improved team collaboration
- Reduced risk of losing code
- Easy identification of changes
- Better software maintenance
- Ability to restore previous versions

Software Configuration Management is a systematic approach for identifying, controlling, tracking, and managing changes and versions of software and related documents throughout the software life cycle.

10. Software Quality Management

10.1 What is Software Quality Management?

Software Quality Management (SQM) is the process of ensuring that software is developed according to **specified requirements, quality standards, and customer expectations.**

Software Quality Management means taking systematic steps to make sure that the software is reliable, correct, efficient, and meets user requirements.

10.2 Objectives of Software Quality Management

The main objectives are:

1. Develop software according to **customer requirements.**
2. Prevent defects as early as possible.
3. Improve software **reliability and performance.**
4. Follow defined **standards and procedures.**
5. Reduce development and maintenance costs.
6. Improve customer satisfaction.
7. Continuously improve the software development process.

10.3 Main Activities of Software Quality Management

Software Quality Management mainly includes:

1. Quality Planning

Quality requirements and standards are identified before development begins.

It defines:

- Quality objectives
- Standards to be followed
- Review and testing activities
- Quality measurements

Example:

Deciding that all critical modules must pass specified test cases before release.

2. Quality Assurance (QA)

Quality Assurance focuses mainly on the **process used to develop software**.

It ensures that the development team follows defined standards, methods, and procedures.

Examples:

- Process audits
- Technical reviews
- Following coding standards
- Process improvement

QA = Focuses on the process

3. Quality Control (QC)

Quality Control focuses mainly on the **software product**.

It identifies defects and checks whether the developed software meets the required quality.

Examples:

- Software testing
- Inspection
- Defect identification
- Product reviews

QC = Focuses on the product

10.4 QA vs QC

Quality Assurance (QA)	Quality Control (QC)
Focuses on process	Focuses on product
Prevents defects	Detects defects
Process-oriented	Product-oriented
Includes audits and process reviews	Includes testing and inspections
Done throughout development	Mainly checks the developed product

10.5 Software Quality Factors

Important software quality factors include:

- **Correctness** – Software performs the required functions.
- **Reliability** – Software works consistently without failure.
- **Efficiency** – Software uses resources effectively.

- **Usability** – Software is easy to learn and use.
- **Maintainability** – Software can be easily modified.
- **Portability** – Software can be moved to another environment.

10.6 Example

Consider an **Online Banking System**.

Quality management ensures that:

- Transactions are performed correctly.
- The system is available and reliable.
- User data is protected.
- The system responds quickly.
- The software is tested before release.
- Development standards are followed.

10.7 Benefits of Software Quality Management

- Better software quality
- Fewer defects
- Improved reliability
- Reduced maintenance cost
- Higher customer satisfaction
- Better compliance with standards
- Improved development process

Software Quality Management is a planned and systematic approach to ensure that software meets specified requirements, follows quality standards, and satisfies customer expectations.

CASE Tools

CASE Tools in Software Project Management

CASE stands for **Computer-Aided Software Engineering**.

CASE tools are software tools that support software engineers and project managers in different activities of software development.

Software Project Management, the focus is on how CASE tools help in **planning, managing, monitoring, documenting and controlling a software project**.

Uses of CASE Tools in Software Project Management

1. Project Planning

- Help managers prepare and organize the software project.
- Help define project activities, tasks and milestones.
- Provide a structured plan for completing the project.

2. Project Scheduling

- Help create project schedules.
- Show the start and completion dates of different activities.
- Help track whether tasks are completed on time.

3. Effort and Cost Estimation

- Help estimate the effort required for the project.
- Support calculation of development cost and time.
- Help managers prepare a project budget.

4. Resource Management

- Help manage project resources such as:
 - Developers
 - Testers
 - Hardware
 - Software
- Help assign resources to different project activities.

5. Project Monitoring

- Help monitor the progress of the project.
- Managers can compare planned work with actual work.
- Help identify delays and problems.

6. Documentation Management

- Help create, store and maintain project documents.
- Examples:
 - Project plans
 - Reports
 - Requirements
 - Design documents
 - Test documents

7. Software Configuration Management

- Help manage different versions of software and project documents.
- Track changes made to files.
- Help maintain version history.

8. Quality Management

- Support software quality-related activities.

- Help maintain quality standards and records.
- Can assist in tracking defects and quality measurements.

9. Team Communication

- Help team members share project information.
- Make project-related documents and reports easily accessible.
- Improve coordination between team members.

10. Integration of Activities

- CASE tools can connect different software engineering activities.
- Information created during one activity can be reused in another activity.
- This reduces duplication of work.

Advantages of CASE Tools in Project Management

- Saves time and effort.
- Reduces manual work.
- Improves project planning.
- Helps monitor project progress.
- Improves documentation.
- Supports resource management.
- Helps control project changes.
- Improves communication among team members.
- Helps maintain consistency of project information.
- Supports better project management decisions.

Example

Suppose a team is developing an **Online Shopping System**.

A CASE tool can help the project manager to:

Plan → Schedule → Assign Resources → Monitor Progress → Manage Changes → Maintain Documentation

For example, the manager can assign developers to the Login, Shopping Cart and Payment modules, set deadlines, monitor their progress and maintain different versions of project documents.

